Tim Pengajar IF2150

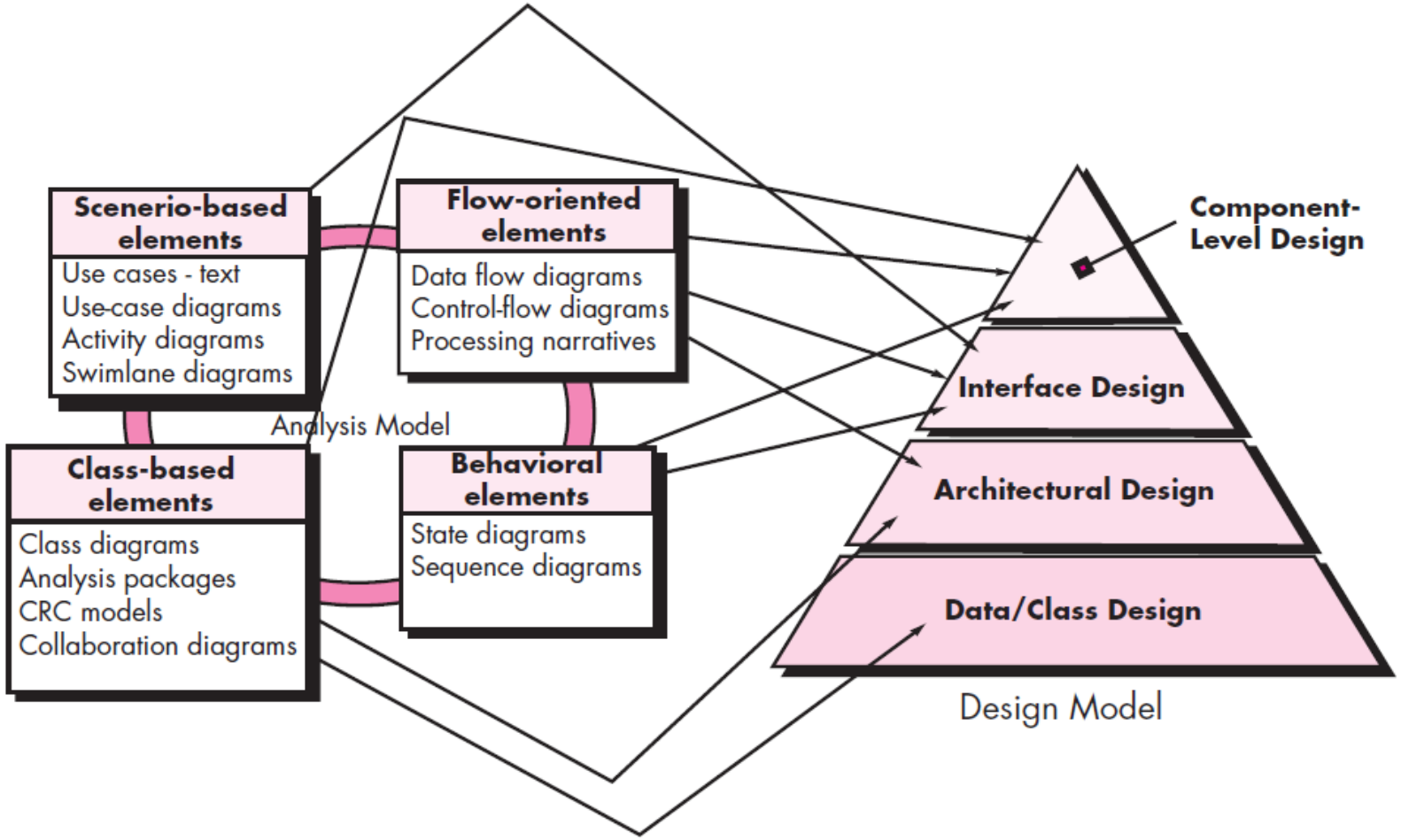
IF2150 – Rekayasa Perangkat Lunak

Architectural Design - 2

SEMESTER I TAHUN AJARAN 2025/2026

KNOWLEDGE & SOFTWARE ENGINEERING

From Analysis to Design



Architectural Views

- **A logical view**, which shows the key abstractions in the system as **objects** or **classes**. It should be possible to relate the system requirements to entities in this logical view.
- **A process view**, which shows how, at run-time, the system is composed of interacting **processes**. This view is useful for making judgments about nonfunctional system characteristics such as performance and availability
- **A development view**, which shows how the software is decomposed for development, that is, it shows the breakdown of the software into **components** that are implemented by a single developer or development team.
- **A physical view**, which shows the system hardware and how software components are distributed across the **processors** in the system. This view is useful for systems engineers planning a system deployment

Component in UML 2.0

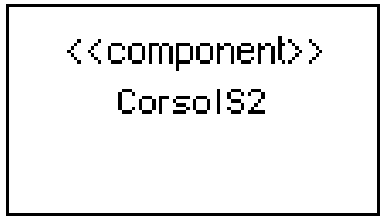
- Modular unit with well-defined interfaces that is replaceable within its environment
- **Autonomous** unit within a system
  - Has one or more provided and required interfaces
  - Its internals are hidden and inaccessible
  - A component is encapsulated
  - Its dependencies are designed such that it can be treated as independently as possible

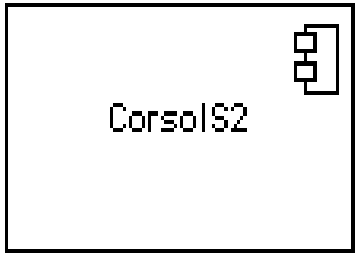
Case Study

- Development of an application collecting students' opinions about courses
- A student can
  - Read
  - Insert
  - Update
  - Make data permanent about the courses in its schedule
- A professor can only see statistic elaboration of the data
- The student application must be installed in pc client
- The manager application must be installed in pc client
- There is one or more servers with DataBase and components for courses management

Component Notation

- A component is shown as a rectangle with
  - A keyword <<component>>
  - Optionally, in the right hand corner a component icon can be displayed
    - A component icon is a rectangle with two smaller rectangles jutting out from the left-hand side
    - This symbol is a visual stereotype
  - The component name





- Components can be labelled with a stereotype

  there are a number of standard stereotypes ex: <<entity>>, <<subsystem>>

Component Elements

- A component can have
  - Interfaces

    An interface represents a declaration of a set of operations and obligations
  - Usage dependencies

    A usage dependency is relationship which one element requires another element for its full implementation
  - Ports

    Port represents an interaction point between a component and its environment
  - Connectors
    - Connect two components
    - Connect the external contract of a component to the internal structure

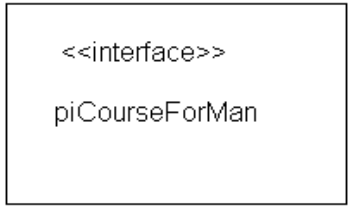
KNOWLEDGE & SOFTWARE ENGINEERING

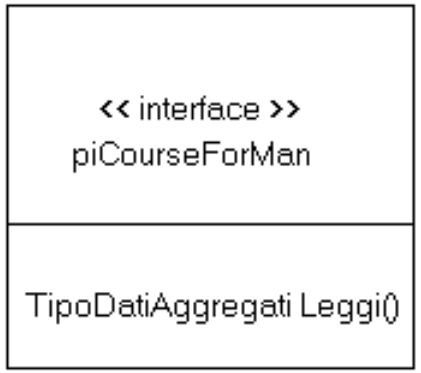
Interface

- A component defines its behaviour in terms of provided and required interfaces
- An interface
  - Is the definition of a collection of one or more operations
  - Provides only the operations but not the implementation
  - Implementation is normally provided by a class/ component
  - In complex systems, the physical implementation is provided by a group of classes rather than a single class

Interface

- May be shown using a rectangle symbol with a keyword <<interface>> preceding the name
- For displaying the full signature, the interface rectangle can be expanded to show details

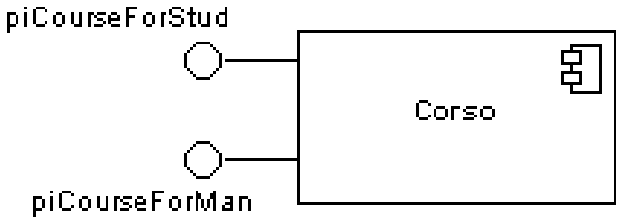


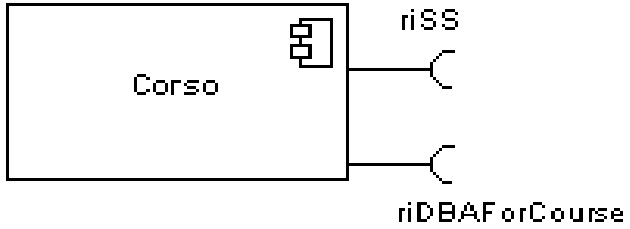


- Can be
  - Provided
  - Required

Interface

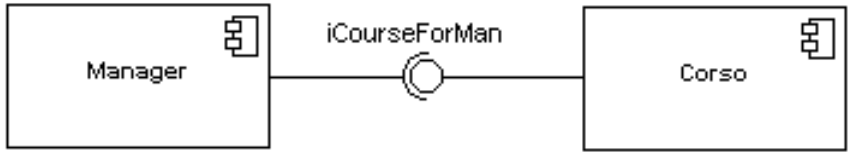
- A provided interface
  - Characterize services that the component offers to its environment
  - Is modeled using a ball, labelled with the name, attached by a solid line to the component
- A required interface
  - Characterize services that the component expects from its environment
  - Is modeled using a socket, labelled with the name, attached by a solid line to the component
  - In UML 1.x were modeled using a dashed arrow





Interface

- Where two components/classes provide and require the same interface, these two notations may be combined

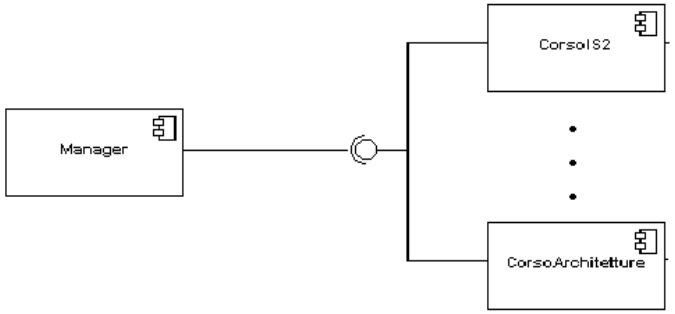


- The ball-and-socket notation hint at that interface in question serves to mediate interactions between the two components
- If an interface is shown using the rectangle symbol, we can use an alternative notation, using dependency arrows

Interface

- In a system context where there are multiple components that require or provide a particular interface, a notation abstraction can be used that combines by joining

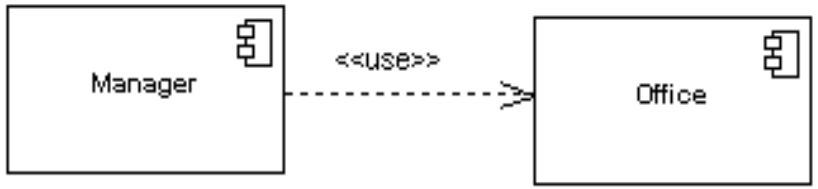
  the interfaces



- A component
  - Specifies a CONTRACT of the services that it provides to its clients and that it requires from others components in terms of its provided and required interfaces
  - Can be replaced
  - The system can be extended

Dependencies

- Components can be connected by usage dependencies

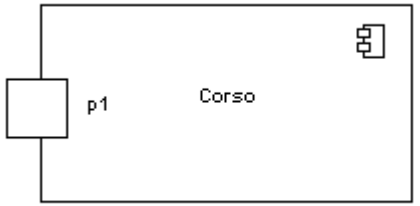


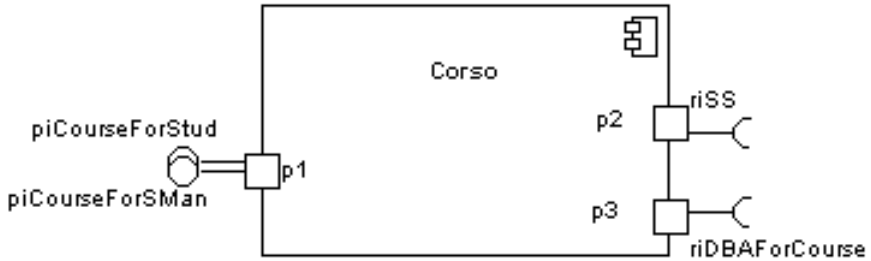
- Usage Dependency
  - A usage dependency is relationship which one element requires another element for its full implementation
  - Is a dependency in which the client requires the presence of the supplier
  - Is shown as dashed arrow with a <<use>> keyword
  - The arrowhead point from the dependent component to the one of which it is dependent

Port

- Specifies a distinct interaction point
  - Between that component and its environment
  - Between that component and its internal parts

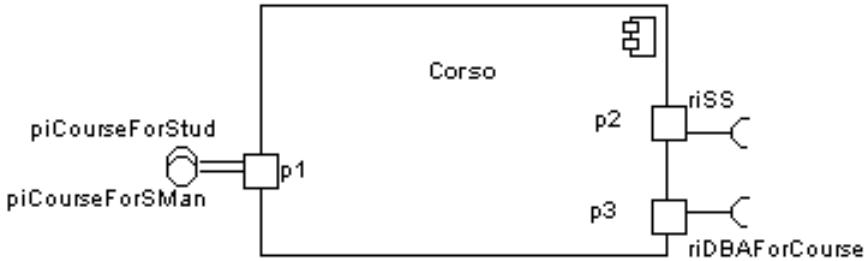
- Is shown as a small square symbol
- Ports can be named, and the name is placed near the square symbol
- Is associated with the interfaces that specify the nature of the interactions that may occur over a port

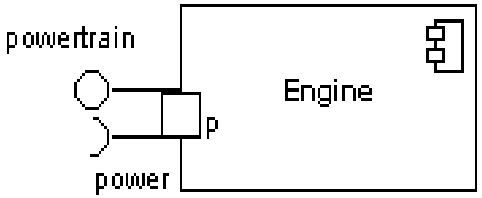




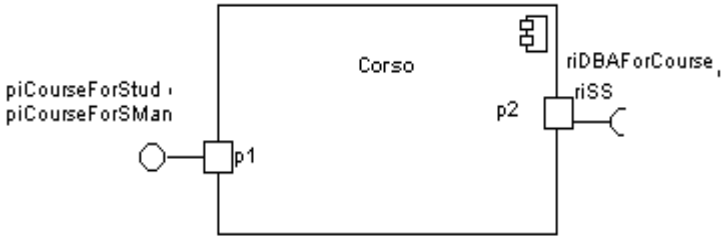
Port

- Ports can support unidirectional communication or bi-directional communication





- If there are multiple interfaces associated with a port, these interfaces may be listed with the interface icon, separated by a commas



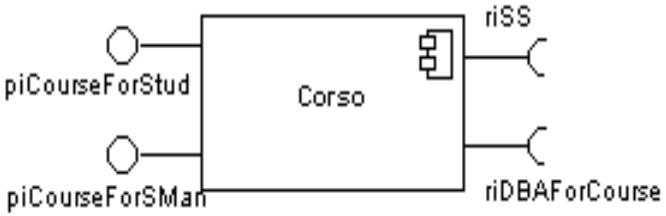
Port

- All interactions of a component with its environment are achieved through a port
- The internals are fully isolated from the environment
- This allows such a component to be used in any context that satisfies the constraints specified by its ports
- Ports are not defined in UML 1.x

External View

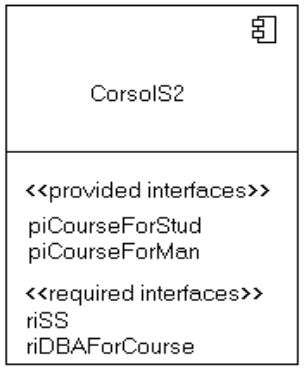
- A component have an external view and an internal view

- An external view (or black box view) shows publicly visible properties and operations



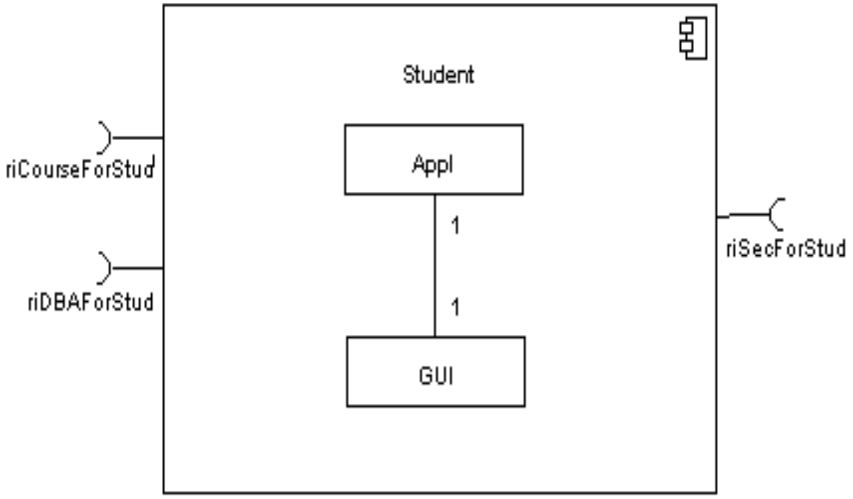
- An external view of a component is by means of interface symbols sticking out of the component box

- The interface can be listed in the compartment of a component box



Internal View

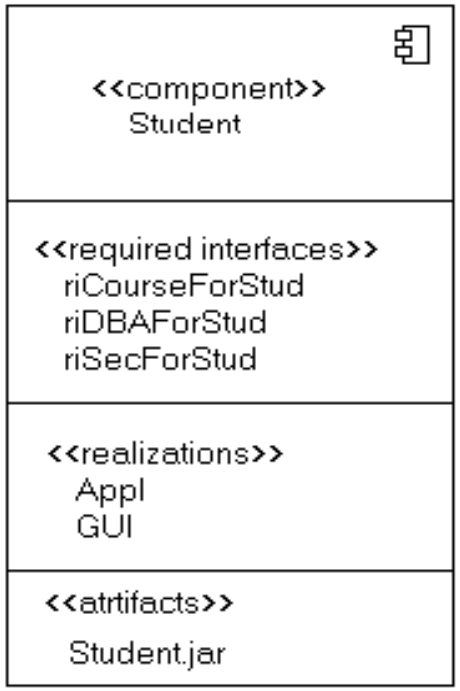
- An internal, or white box view of a component is where the realizing classes/components are nested within the component shape



- Realization is a relationship between two set of model elements
  - One represents a specification
  - The other represent an implementation of the latter

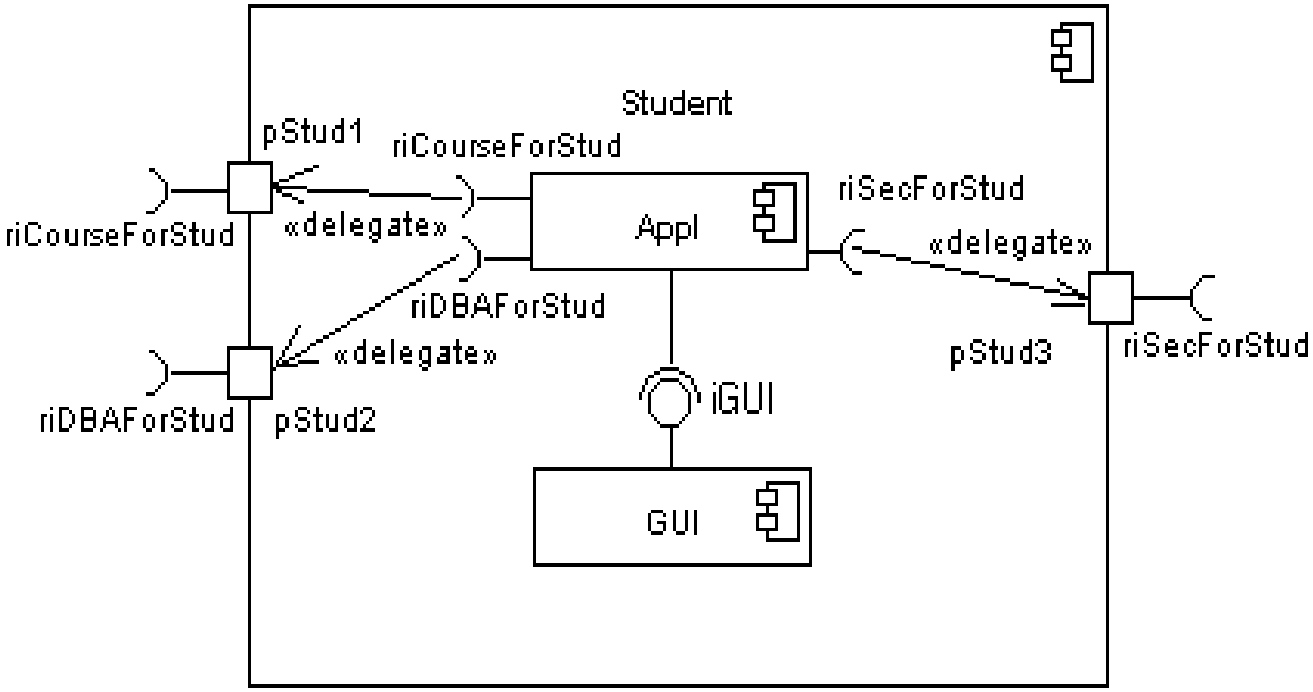
Internal View

- The internal class that realize the behavior of a component may be displayed in an additional compartment
  - Compartments can also be used to display parts, connectors or implementation artifacts
  - An artifact is the specification of a phisycal piece of information



Internal View

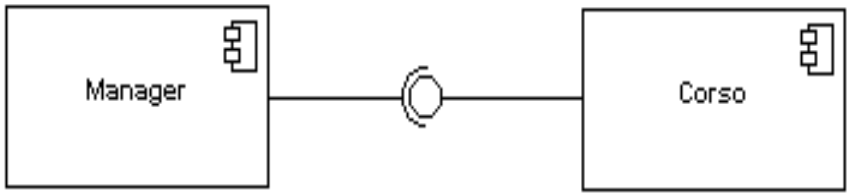
- Components can be built recursively



Assembly

- Two kinds of connectors:
  - Delegation
  - Assembly
- ASSEMBLY CONNECTOR
  - A connector between 2 components defines that one component provides the services that another component requires
  - He must only be defined from a required interface to a provided interface
  - An assembly connector is notated by a "ball-and-socket" connection

This notation allows for succint grafical wiring of components

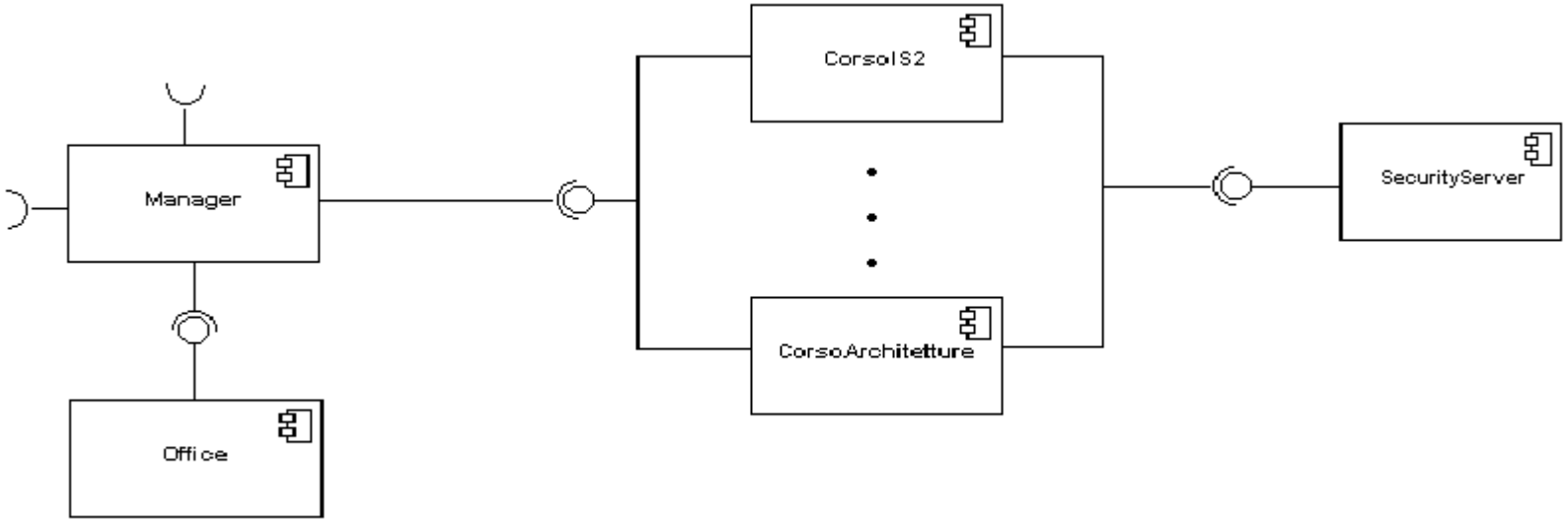


Semantics

- The semantics for an assembly connector :
  - Are that signals travel along an instance of a connector originating in a required port and delivered to a provided port
  - The interfaces provided and required must be compatible
  - The interface compatibility between provided and required ports that are connected enables an existing component in a system to be replaced

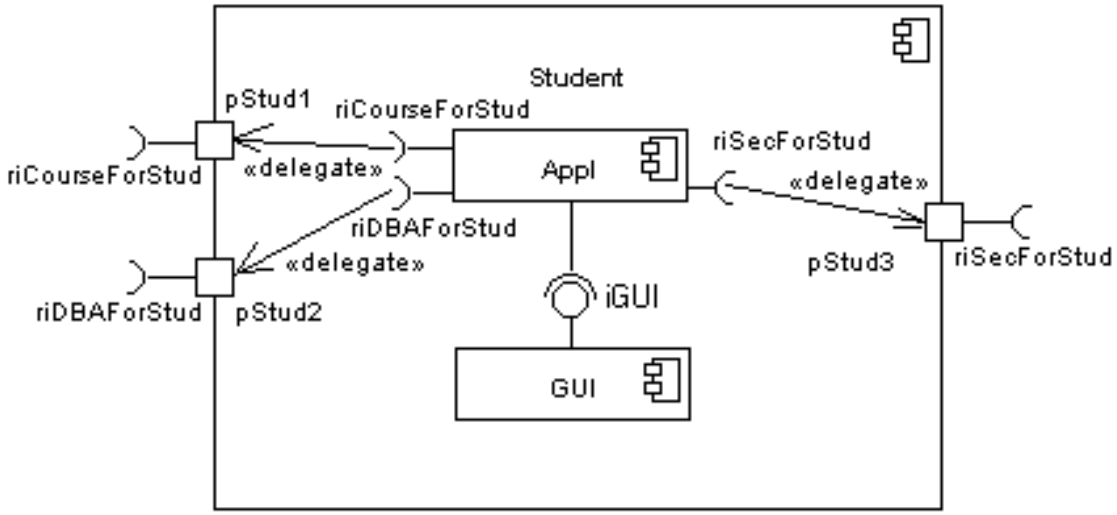
Semantics

- Multiple connections directed from a single required interface to provided interfaces indicates that the instance that will handle the signal will be determined at execution time



Delegation

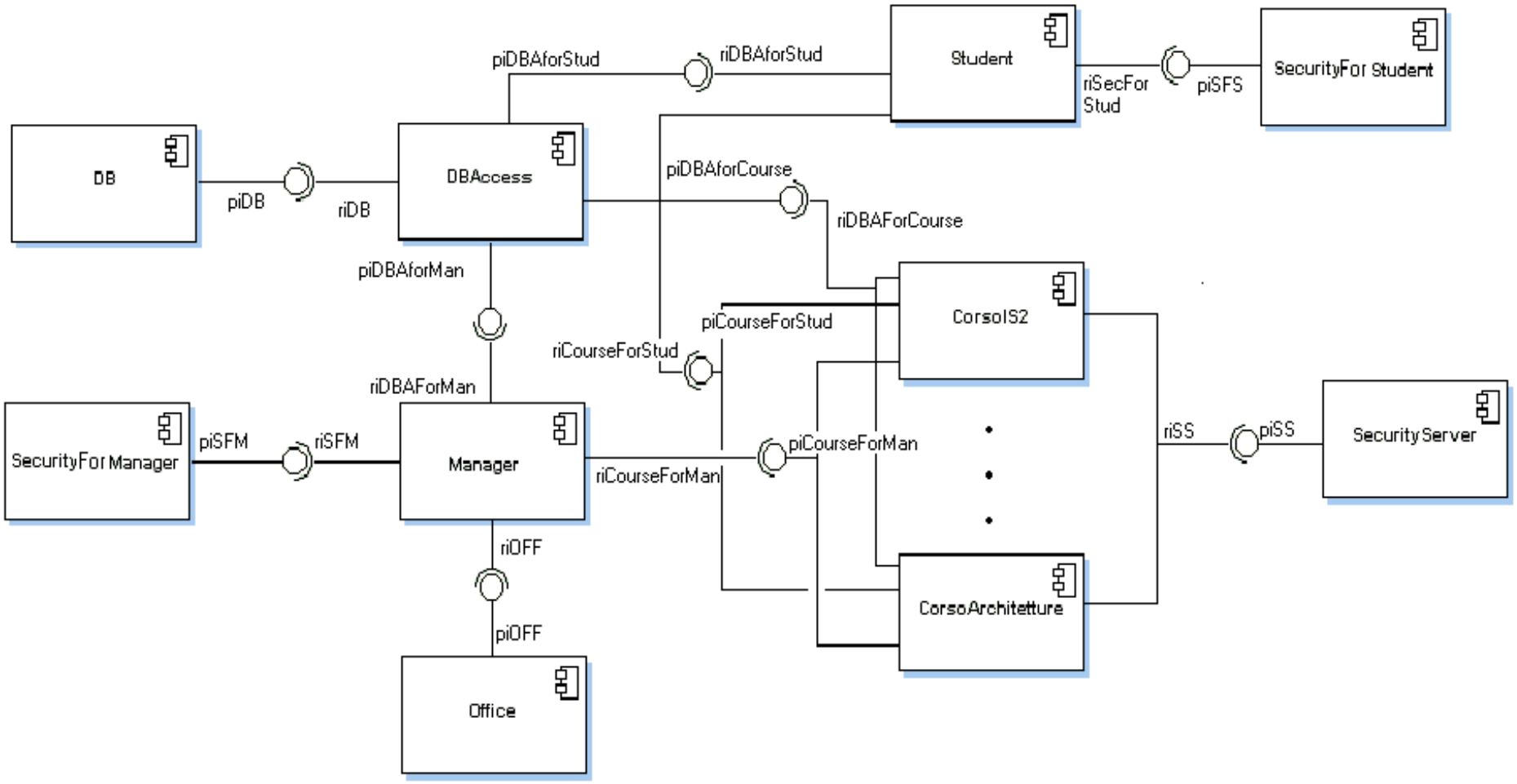
- Delegation Connector
  - Links the external contract of a component to the internal realization
  - Represents the forwarding of signals
  - He must only be defined between used interfaces or ports of the same kind



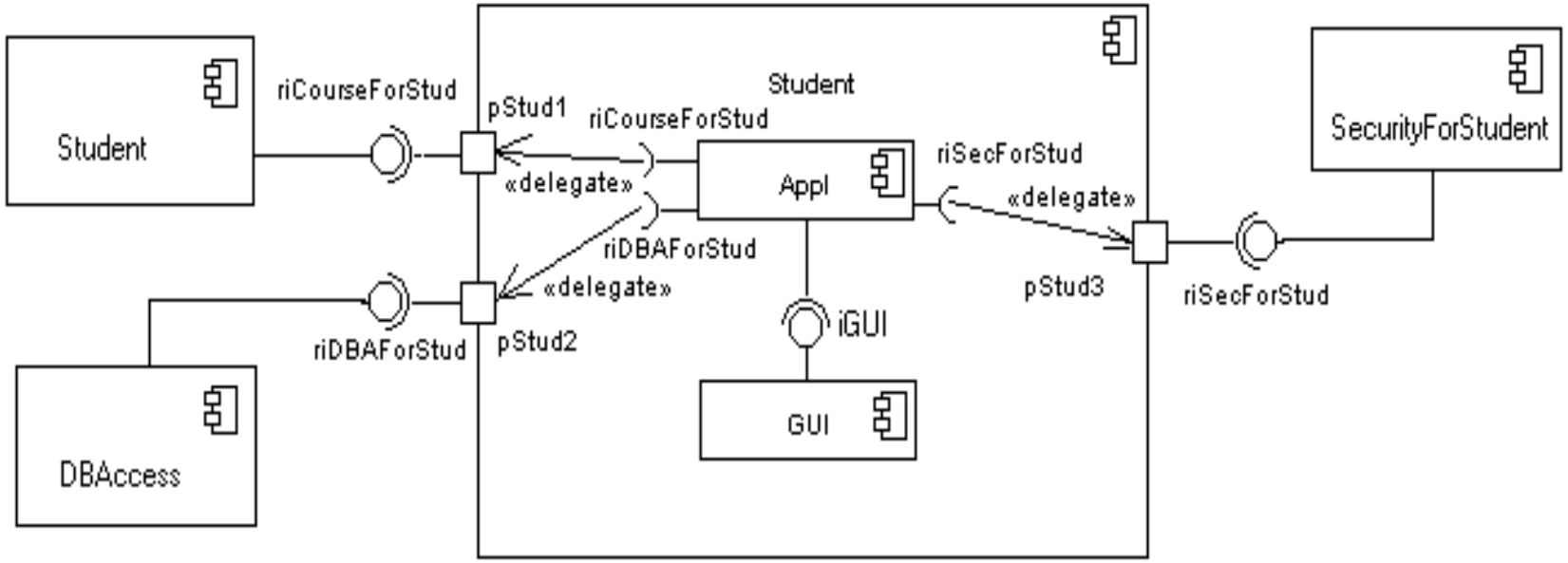
Delegation

- The target interface must support a signature compatible with a subset of operations of the source interface
- A port may delegate to a set of ports on subordinate components
- The union of the target interfaces must be signature compatible with the source interface
- Semantics:
  - Is a declaration that behaviour that is available on a component instance is not realized by that component itself, but by another instance that has compatible capabilities
  - Is used to model the hierarchical decomposition
  - Message and signal flow will occur between the connected ports

Case Study


Student
SecurityFor Student
riSecFor Stud
piSFS
piDBAforStud
riDBAforStud
DB
piDB
riDB
DBAccess
piDBAforCourse
riDBAForCourse
piDBAforMan
CorsoIS2
piCourseForStud
riCourseForStud
riDBAForMan
SecurityFor Manager
piSFM
riSFM
Manager
riCourseForMan
piCourseForMan
riSS
piSS
Security Server
riOFF
CorsoArchitetture
piOFF
Office

Case Study


Student
riCourseForStud
pStud1
Student
riCourseForStud
«delegate»
Appl
riSecForStud
«delegate»
SecurityForStudent
riDBAForStud
«delegate»
pStud3
riSecForStud
riDBAForStud
pStud2
iGUI
GUI
DBAccess

Aktivitas — Memahami Komponen dalam UML 2.0

- Instruksi:
  - Pilih satu contoh sistem kecil (misalnya student course feedback system pada case study).
  - Identifikasi minimal 3 komponen (misalnya: Student Client, Professor App, Database Server).
  - Tentukan:
    - Komponen mana yang memiliki provided interface (ball notation) dan required interface (socket notation)
    - Apa contract layanan antar komponen tersebut?
- Output: Sketsa diagram komponen sederhana menggunakan simbol <<component>> dan notasi ball-and-socket.

Deployment Diagrams

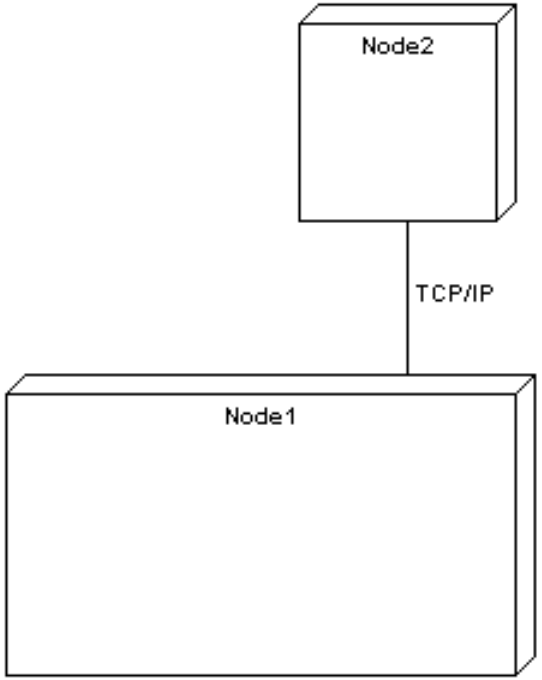
- There is a strong link between components diagrams and deployment diagrams

- Deployment diagrams
  - Show the physical relationship between hardware and software in a system
  - Hardware elements:
    - Computers (clients, servers)
    - Embedded processors
    - Devices (sensors, peripherals)
  - Are used to show the nodes where software components reside in the run-time system

KNOWLEDGE & SOFTWARE ENGINEERING

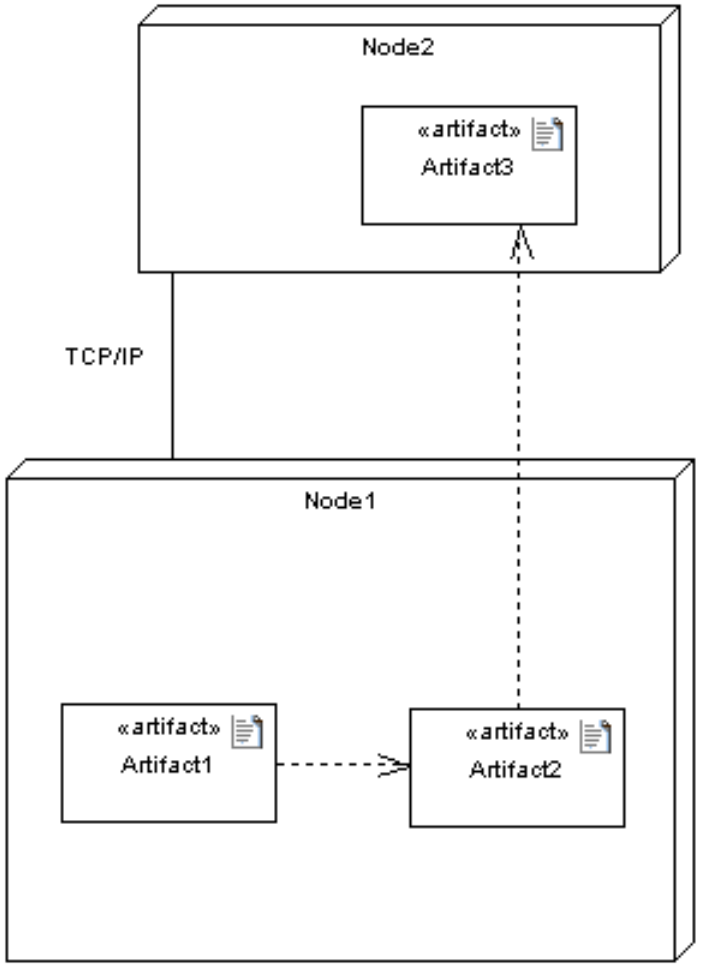
Deployment Diagrams

- Deployment diagram
  - Contains nodes and connections
  - A node usually represent a piece of hardware in the system
  - A connection depicts the communication path used by the hardware to communicate
  - Usually indicates the method such as TCP/IP



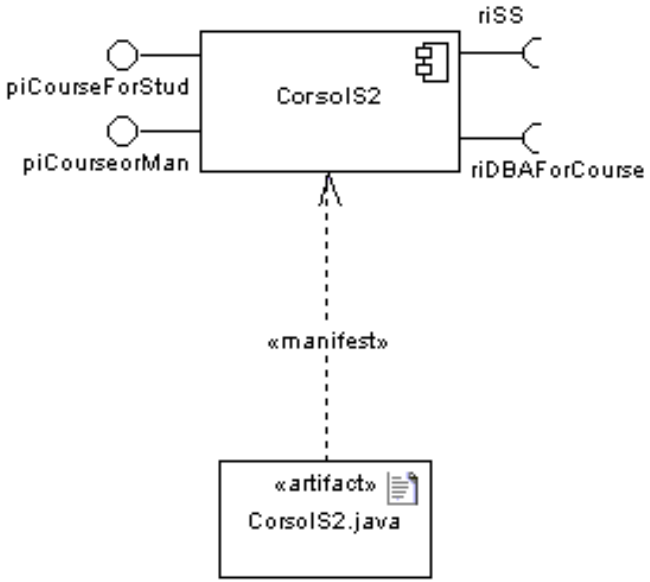
Deployment Diagrams

- Deployment diagrams contain artifact
- An artifact
  - Is the specification of a phisycal piece of information
  - Ex: source files, binary executable files, table in a database system,....
  - An artifact defined by the user represents a concrete element in the physical world

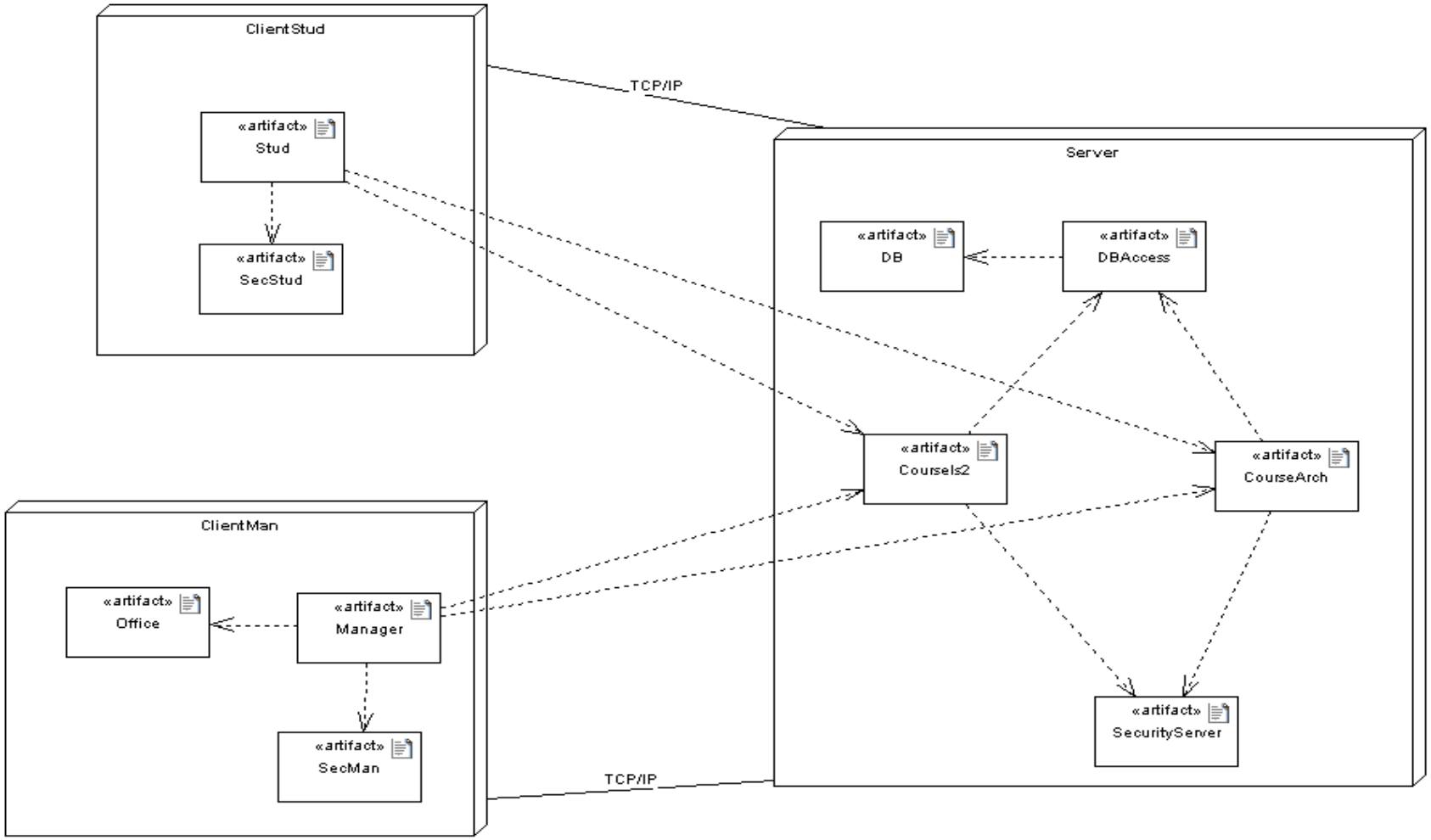


Deployment Diagrams

- An artifact manifest one or more model elements
- A <<manifestation>> is the concrete physical of one or more model elements by an artifact
- This model element often is a component
- A manifestation is notated as a dashed line with an open arrow-head labeled with the keyword <<manifest>>



Deployment Diagrams



Reference

- COMPONENT DIAGRAM in UML 2.0, Veronica Carrega
- UML 2.0 Superstructure Specification

  August 2, 2003

  UML 2 Superstructure Final Adopted Specification

  www.omg.org/cgi-bin/doc?ptc/2003-08-02
- The Diagrams of UML 2.0

  by Scott W. Ambler, 2003-2004

  www.agilemodeling.com/essays/umlDiagrams.htm
- UML overview

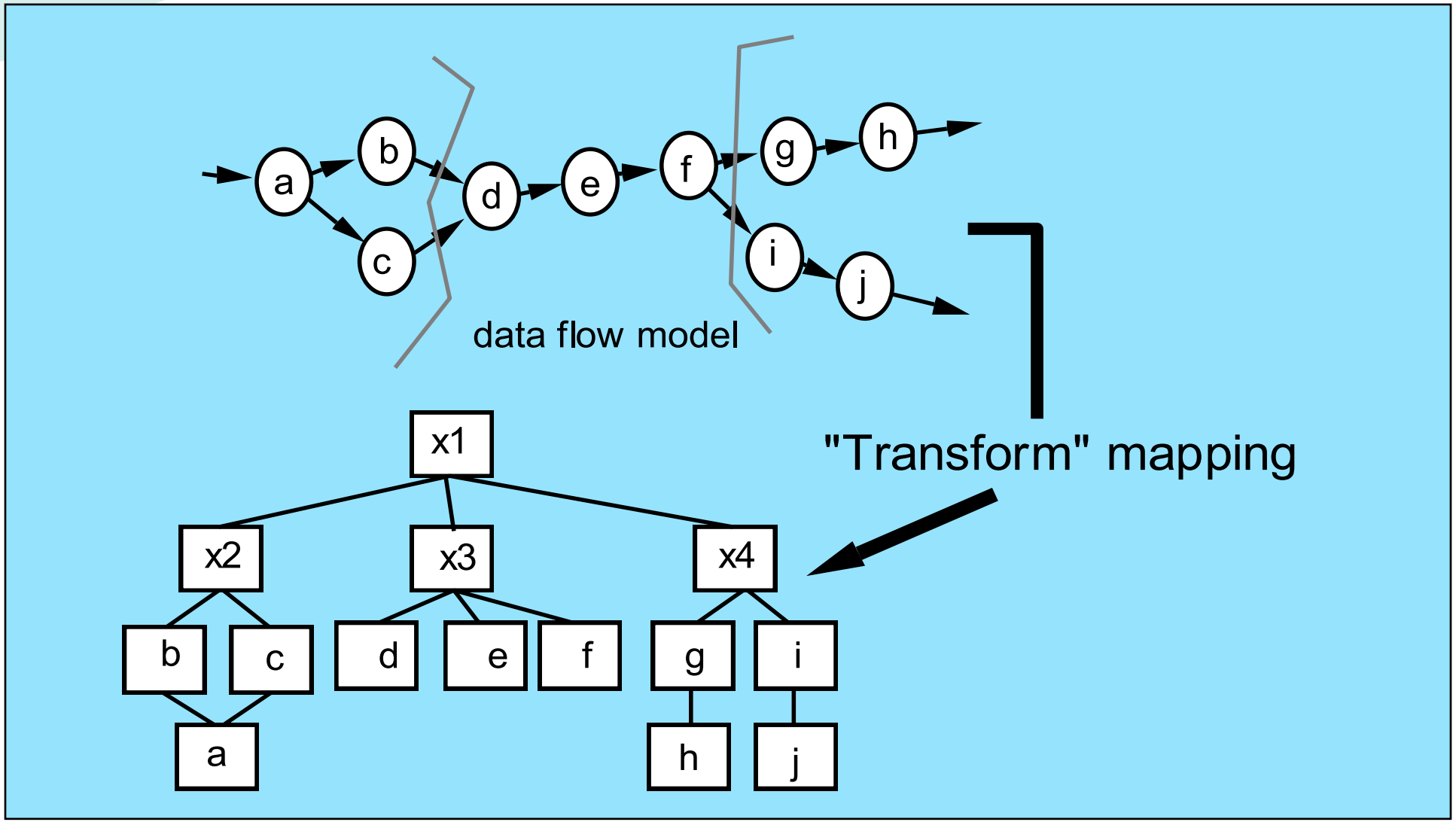
  By Mandar Chitnis, Pravin Tiwari, & Lakshmi Ananthamurthy

  http://www.developer.com/design/article.php/1553851

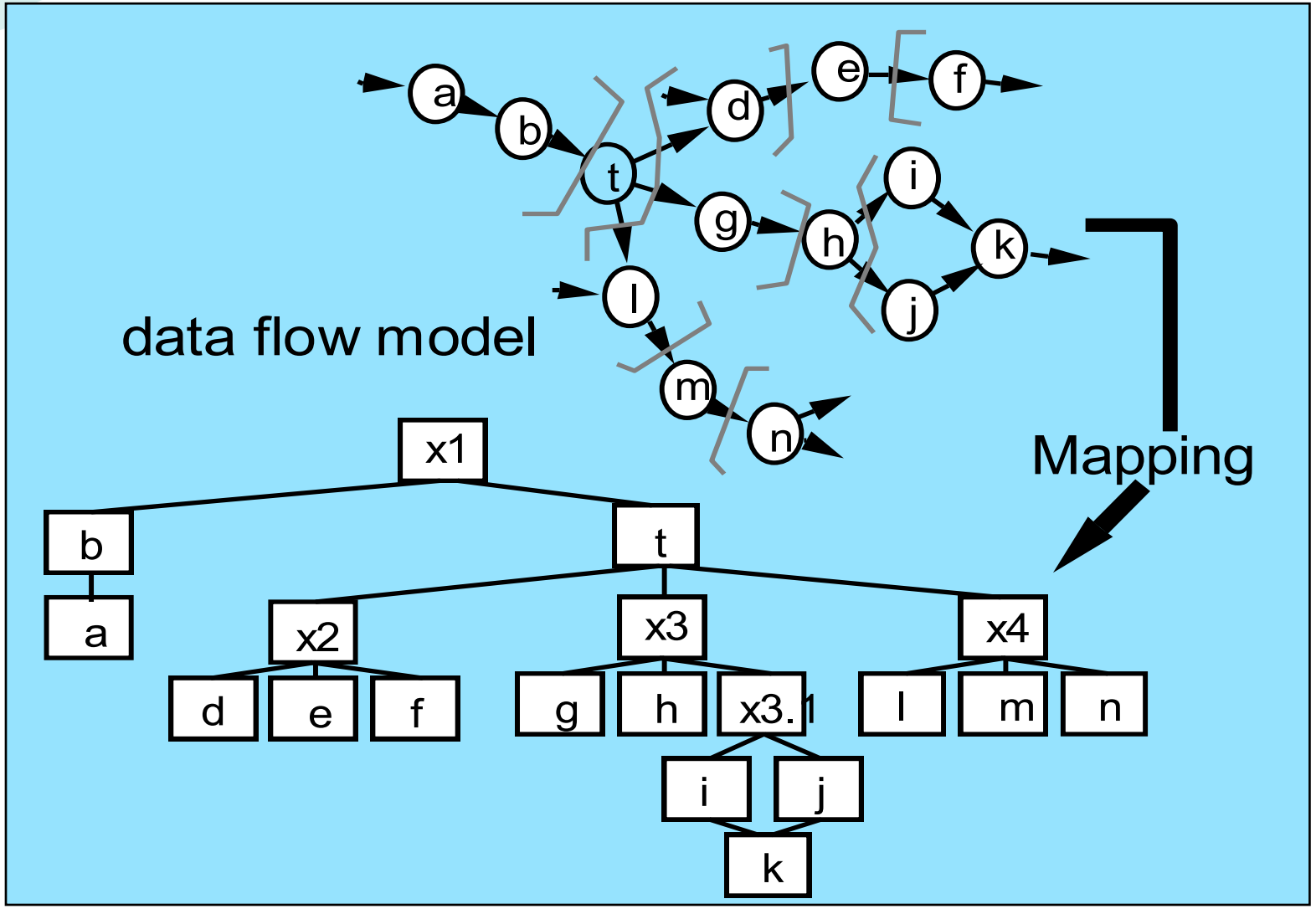
Architectural Mapping using Data Flow

- A mapping technique, called *structured design*, is often characterized as a data flow-oriented design method because it provides a convenient transition from a data flow diagram to software architecture
- Six steps process:
  1. the type of information flow is established,
  2. flow boundaries are indicated,
  3. the DFD is mapped into the program structure,
  4. control hierarchy is defined,
  5. the resultant structure is refined using design measures and heuristics, and
  6. the architectural description is refined and elaborated

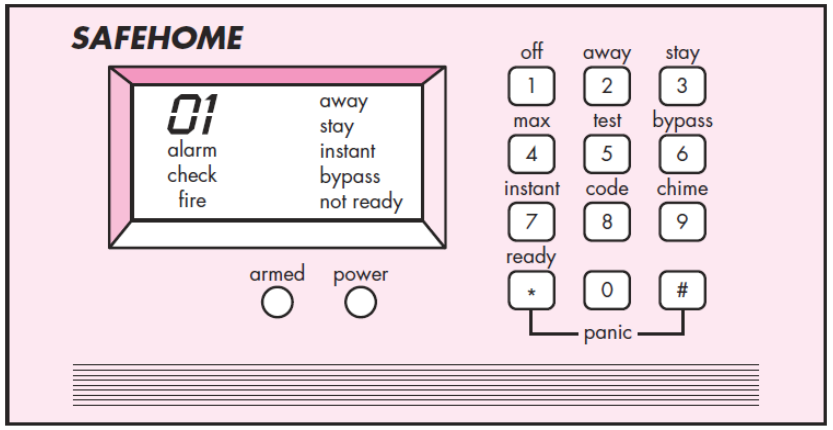
Transform Mapping

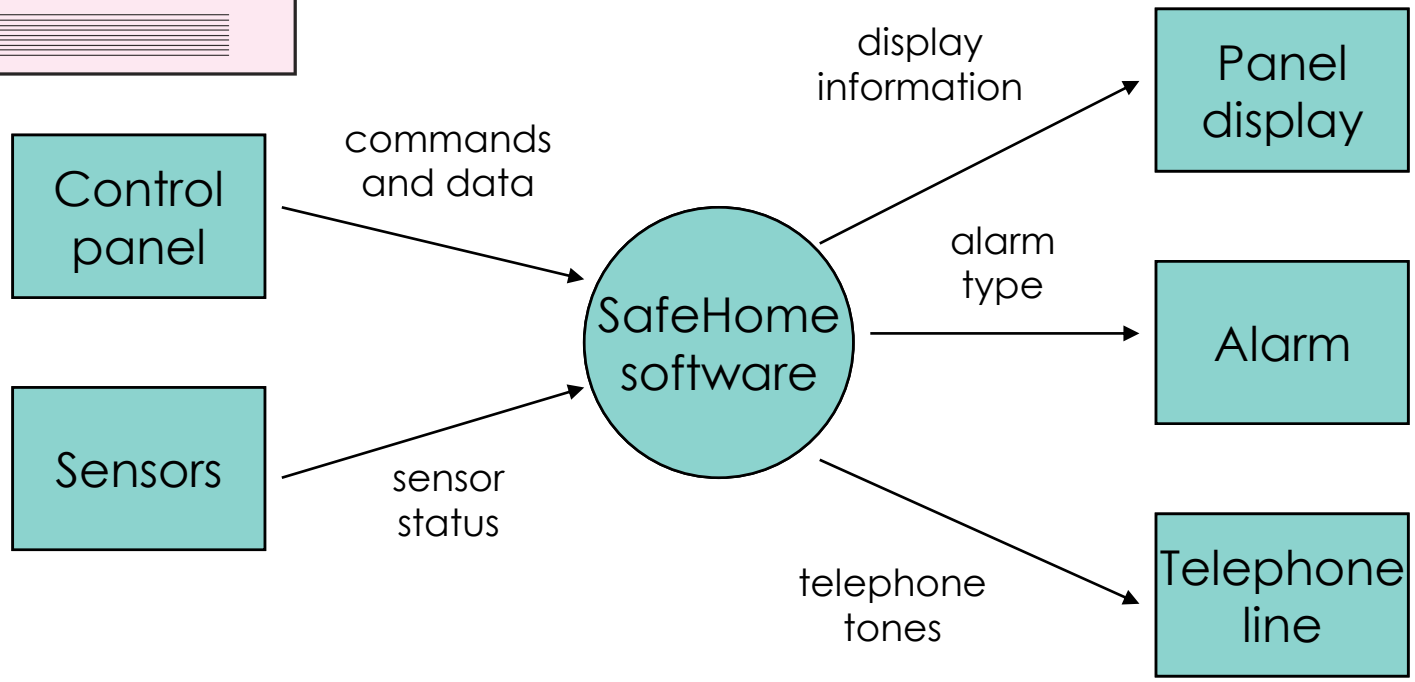


Transaction Mapping

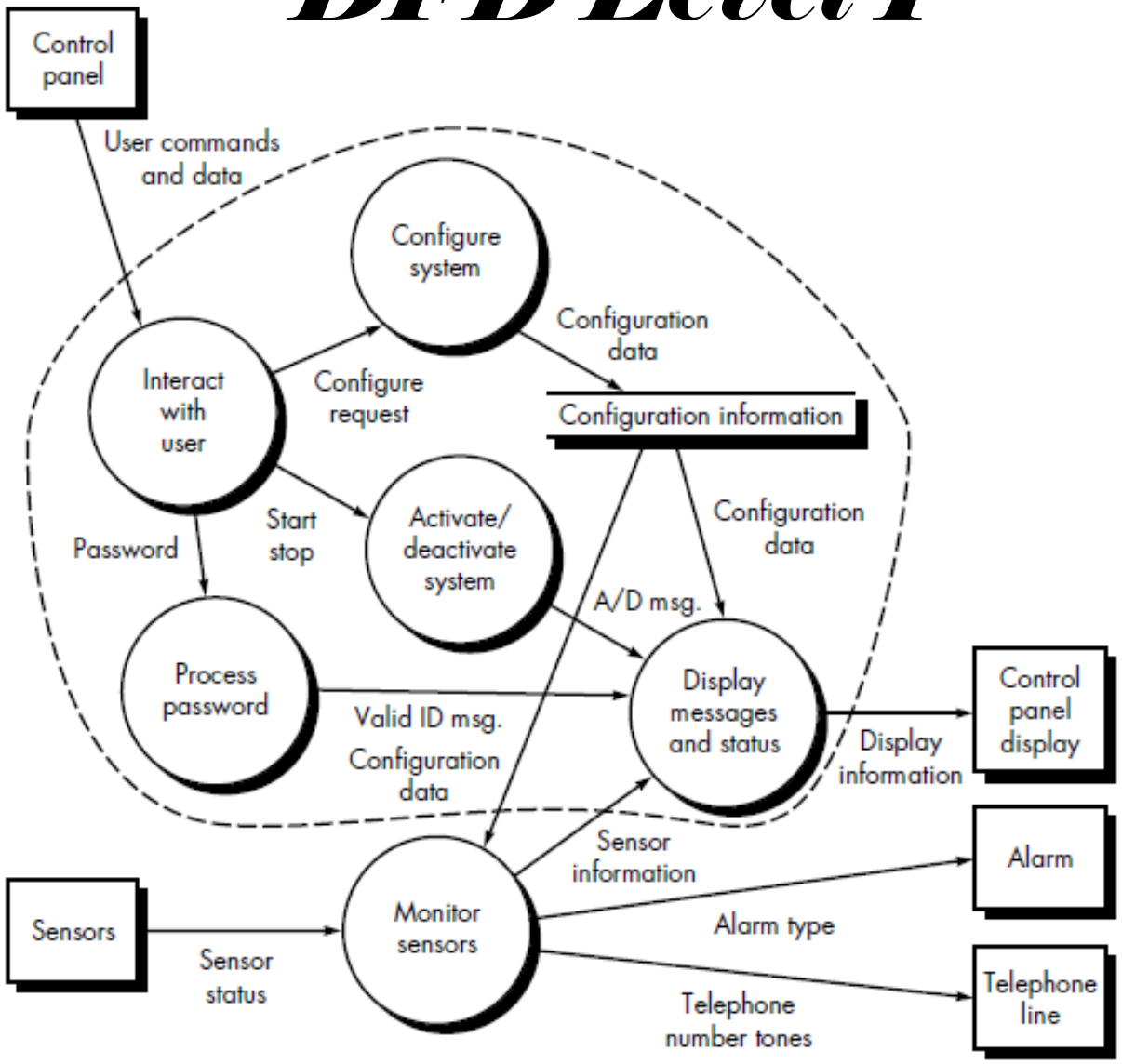


SafeHome Software Context Model

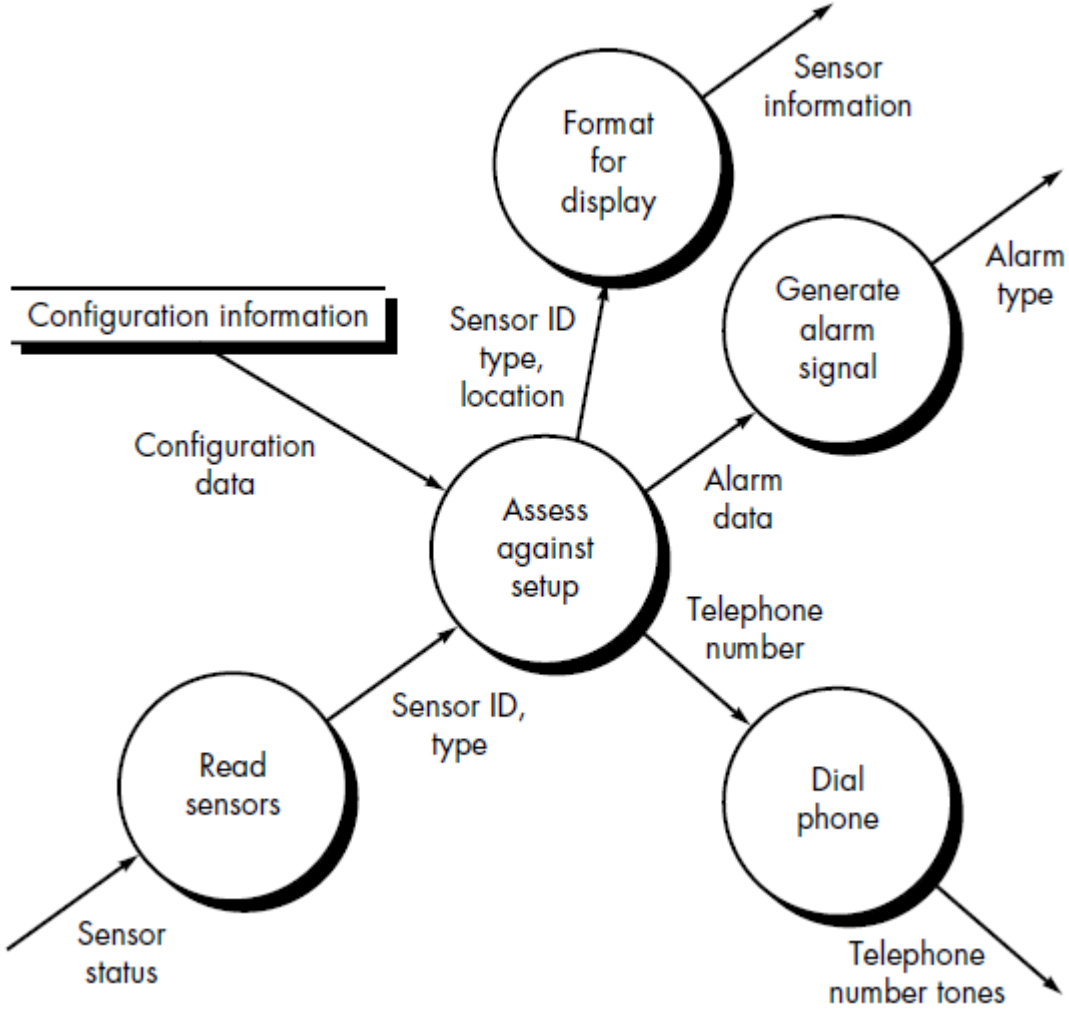




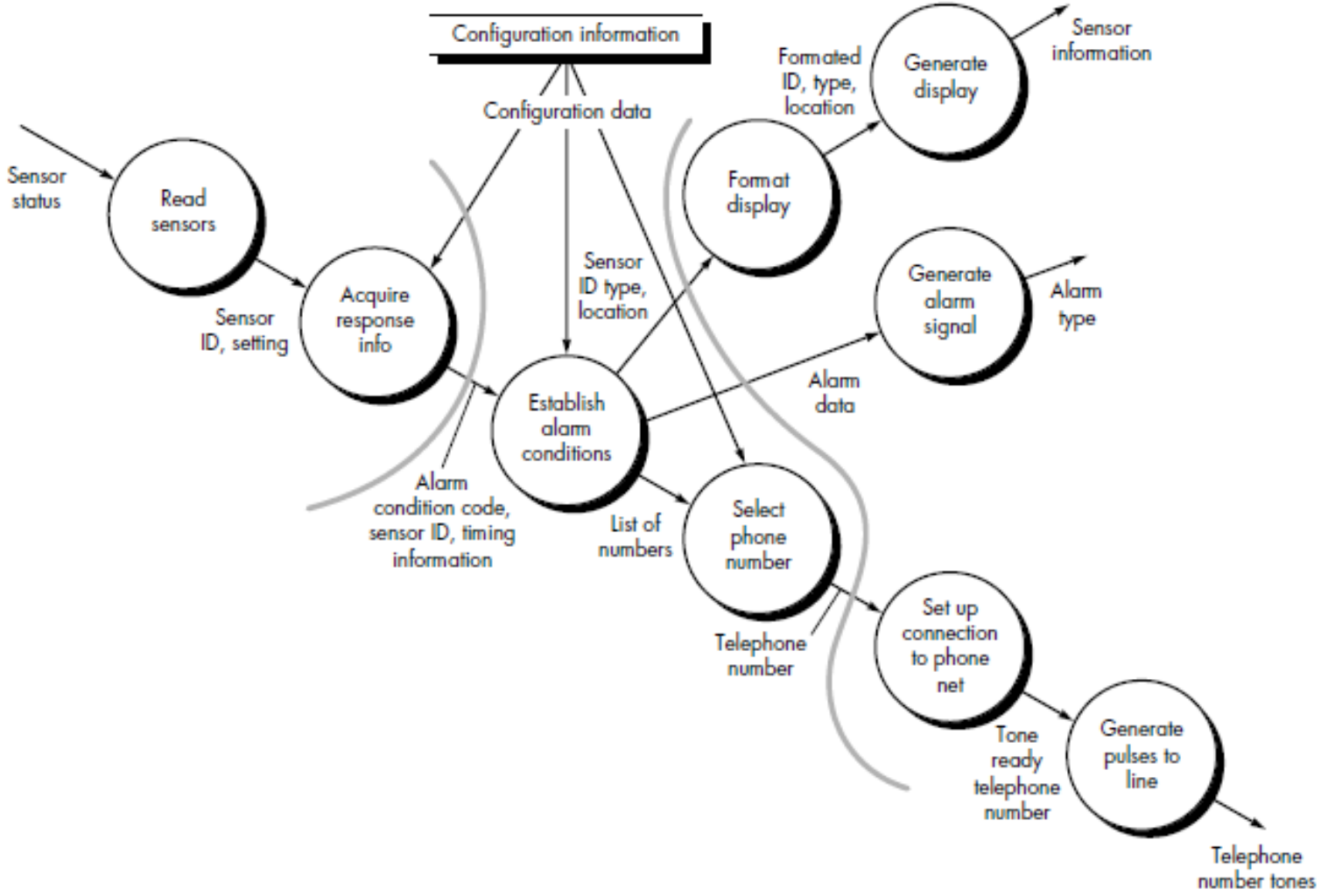
DFD Level 1


Control panel
User commands and data
Configure system
Configuration data
Interact with user
Configure request
Configuration information
Activate/ deactivate system
Start stop
Password
Configuration data
A/D msg.
Process password
Display messages and status
Control panel display
Valid ID msg.
Configuration data
Display information
Sensor information
Alarm
Sensors
Monitor sensors
Alarm type
Sensor status
Telephone line
Telephone number tones

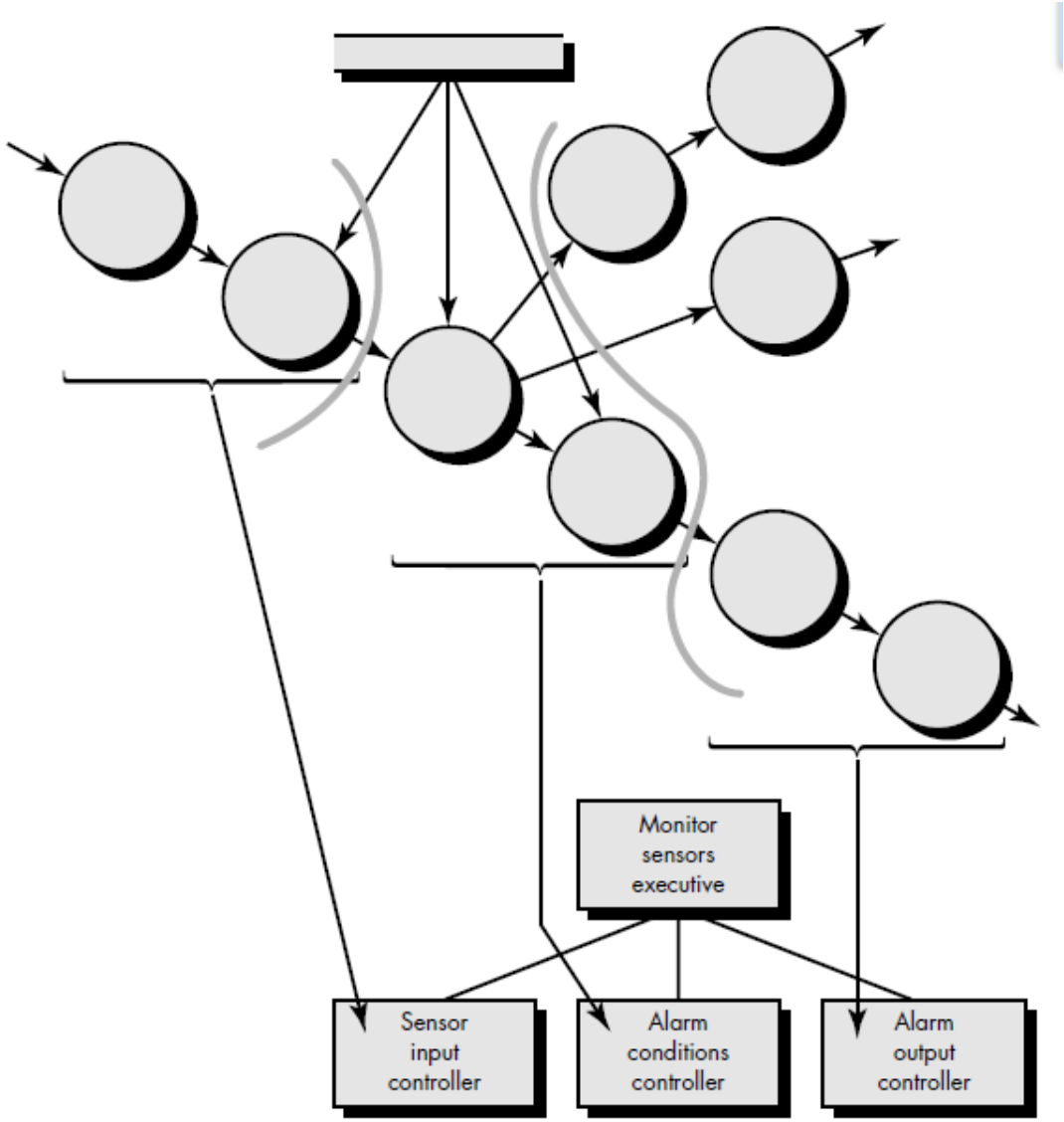
DFD Level 2 "Monitor Sensor"



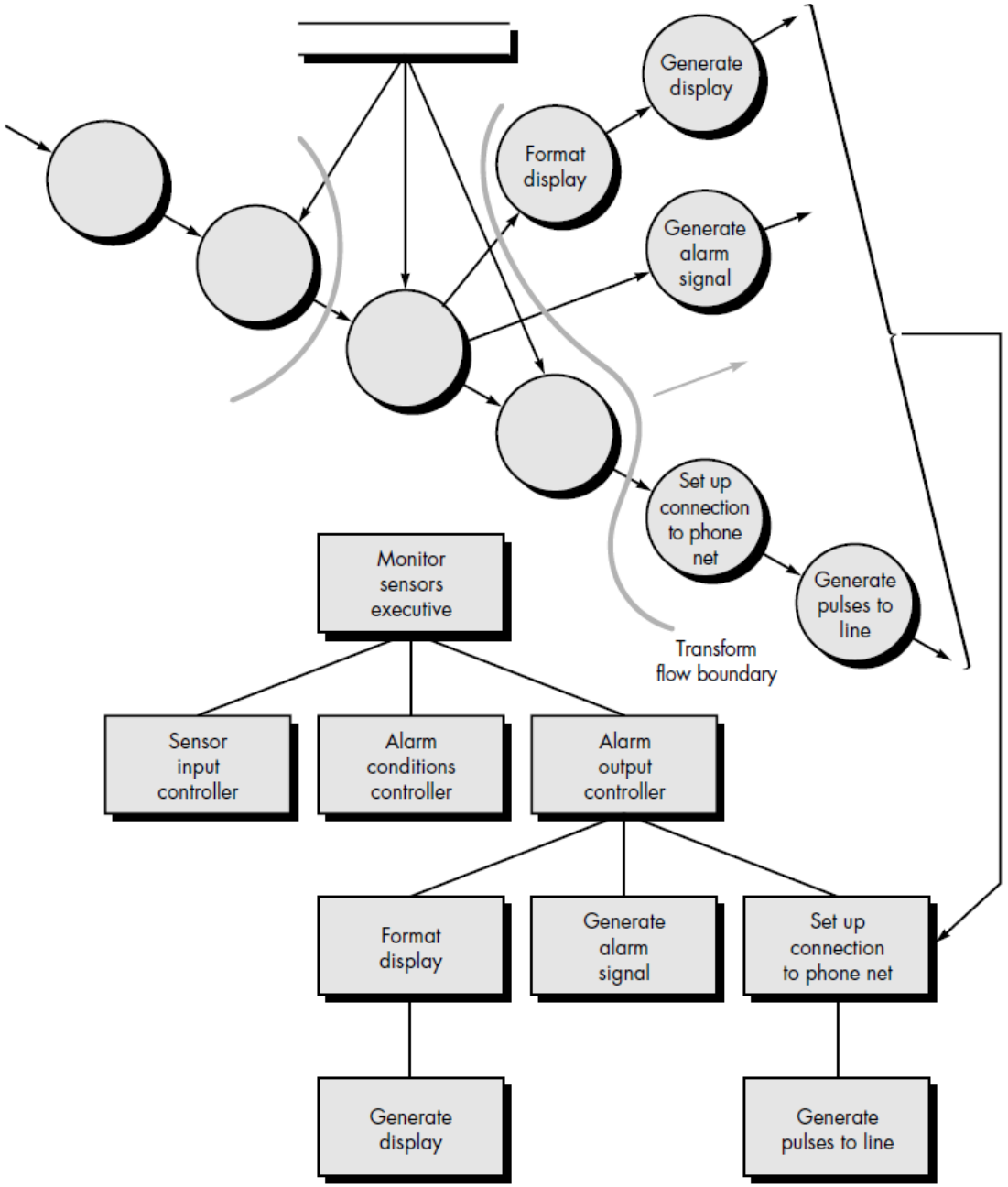
DFD level 2 'monitor sensor' with flow boundary



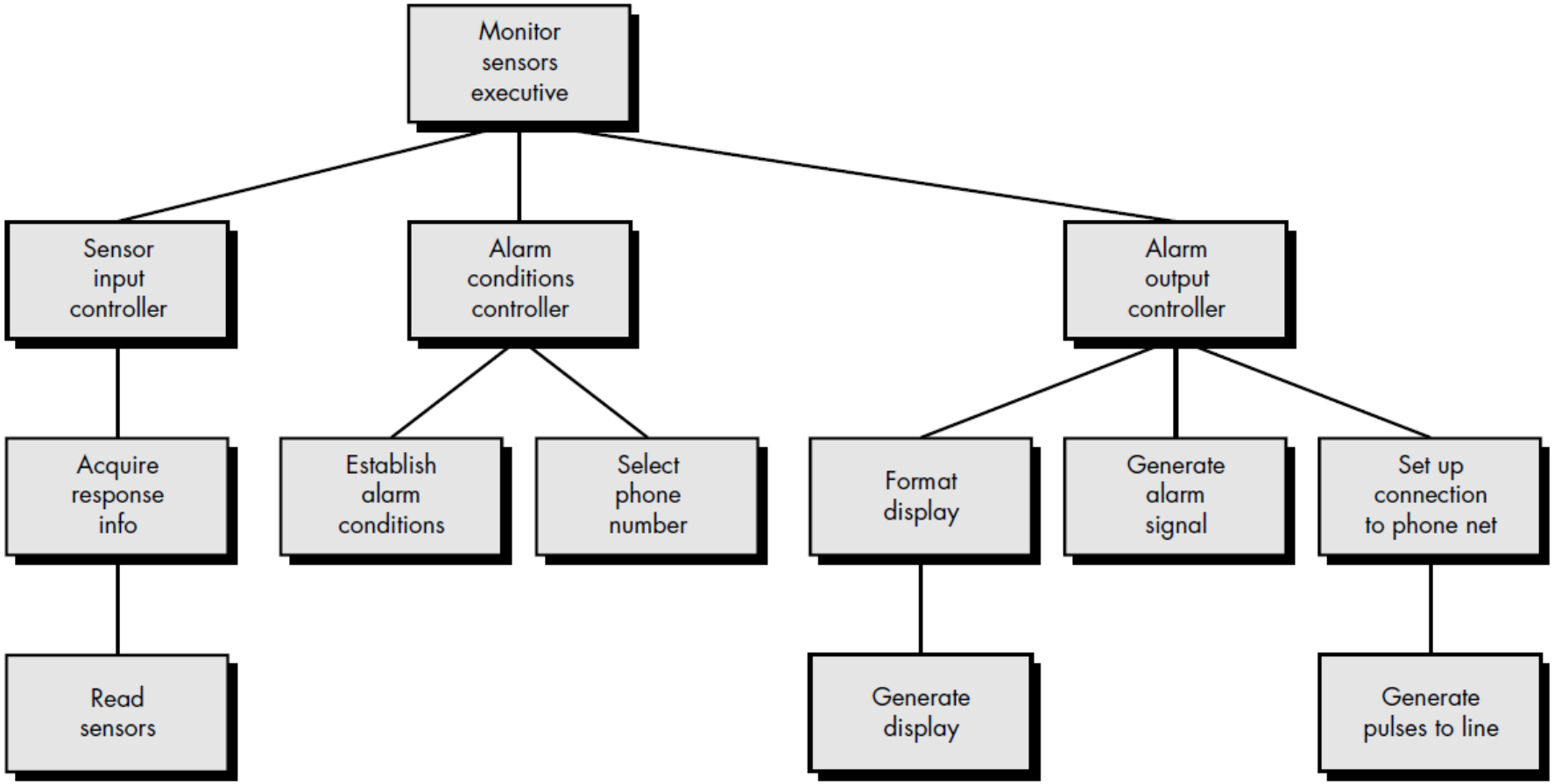
First level factoring for "monitor sensor"



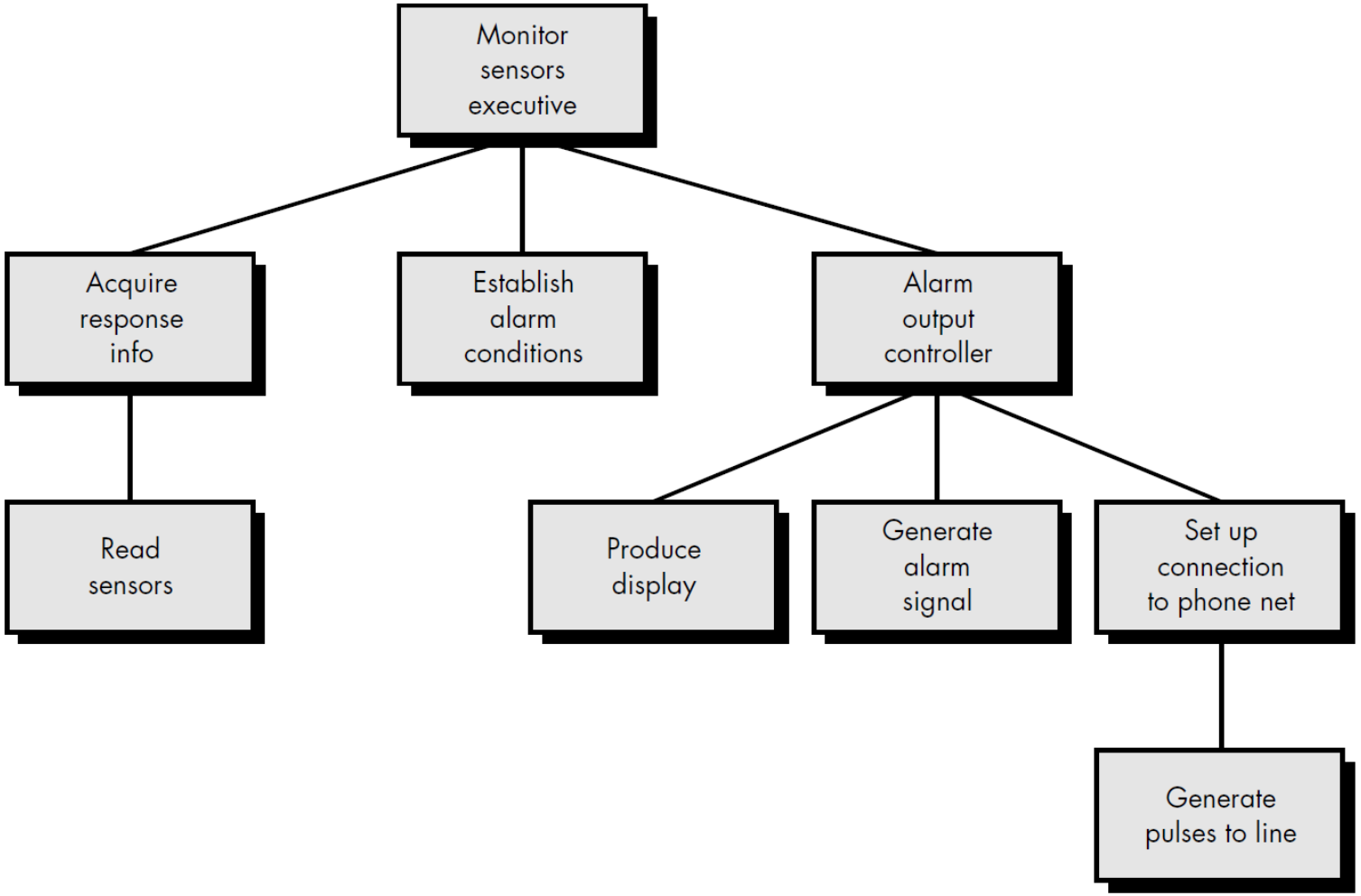
Second level factoring



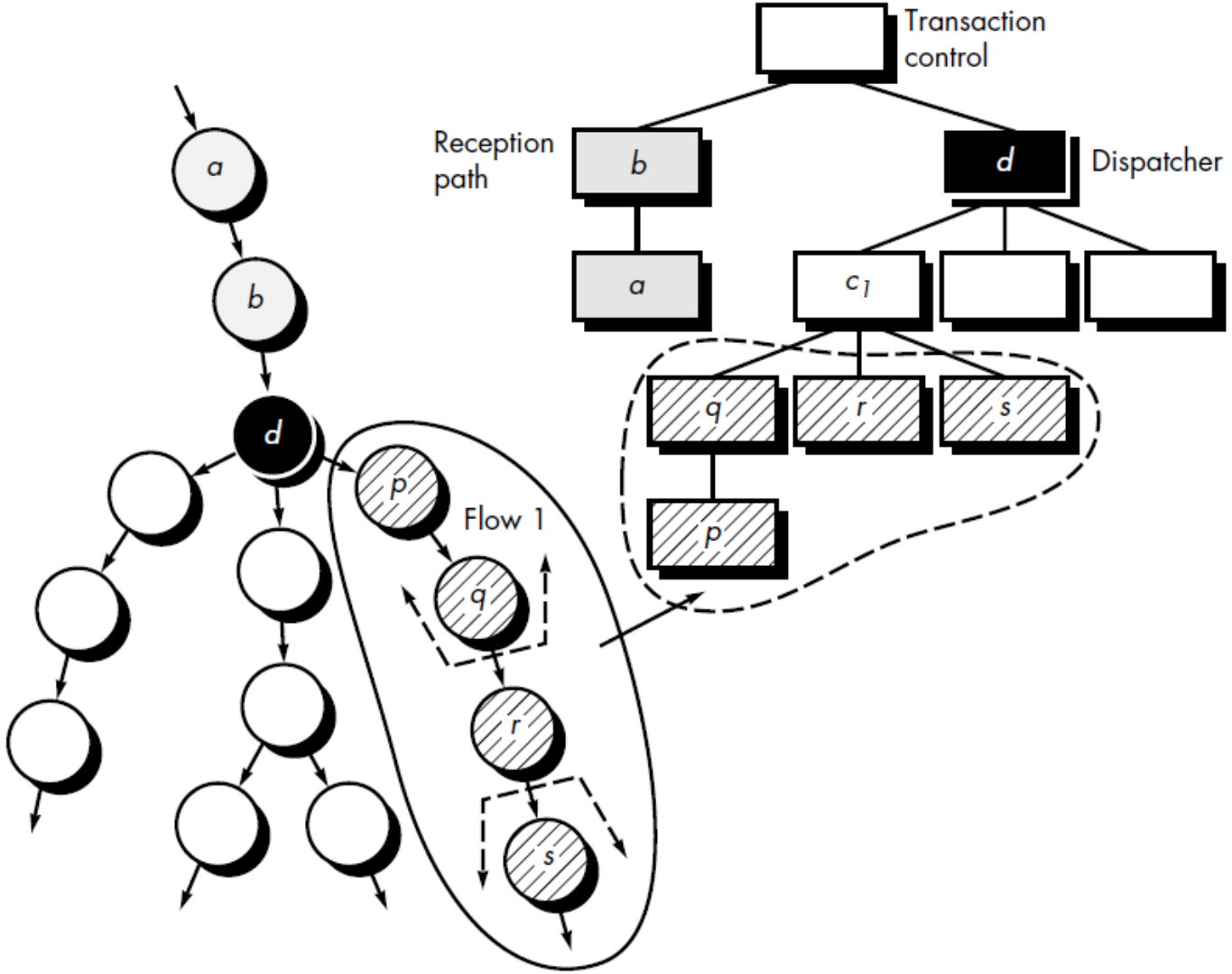
Structured Chart (SC) for SafeHome (first iteration)



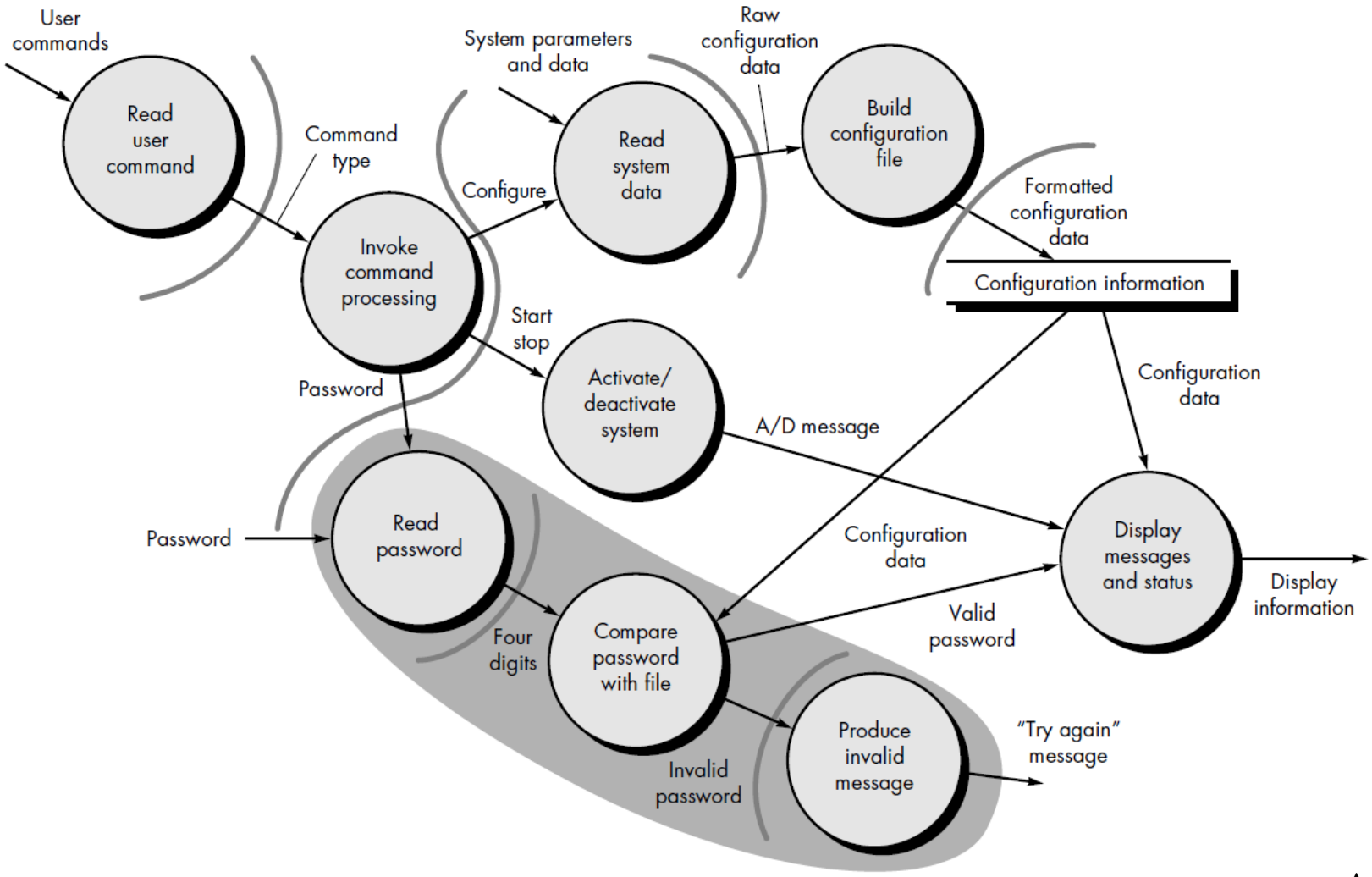
Structured Chart (SC) for SafeHome (second iteration)



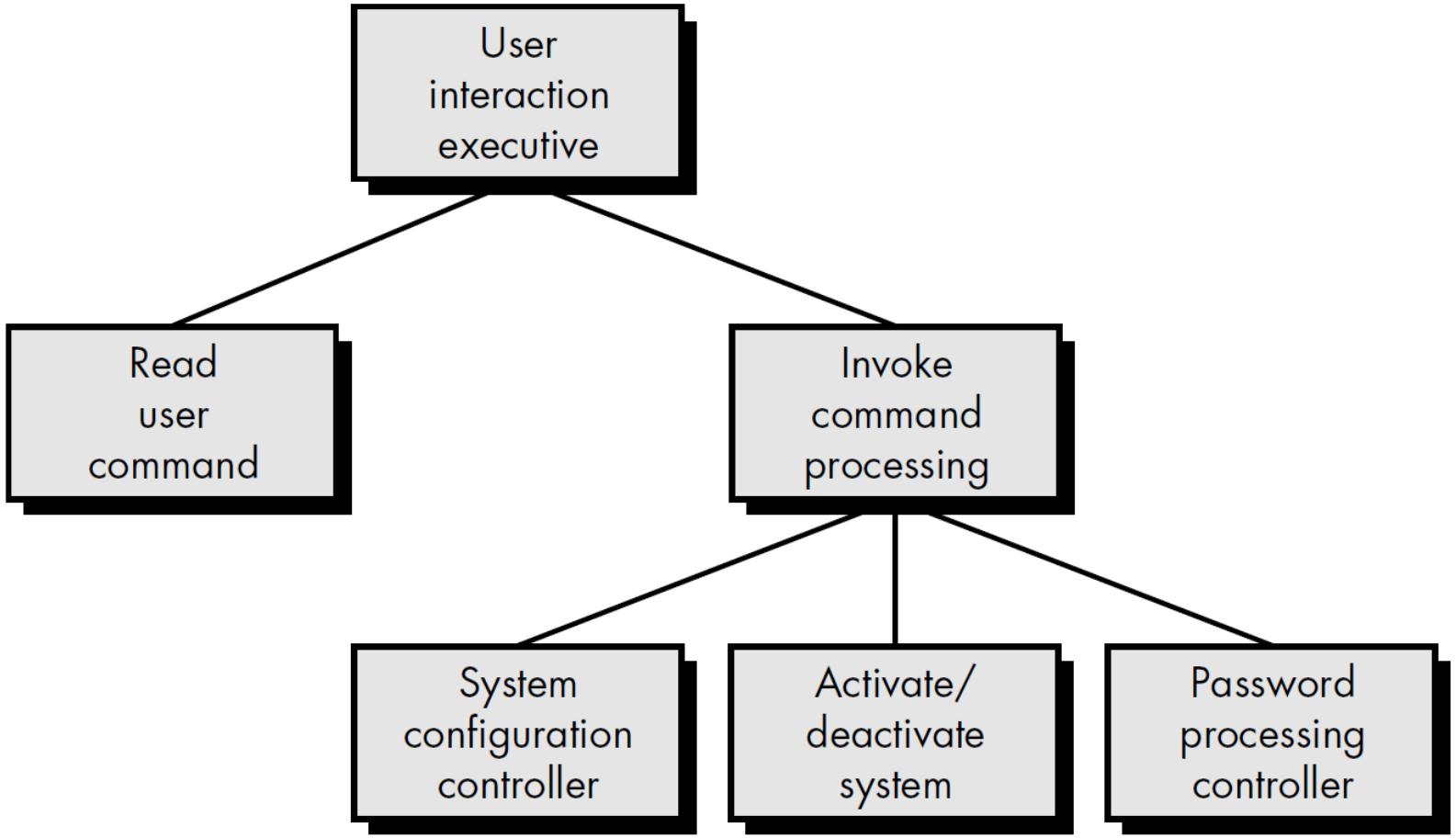
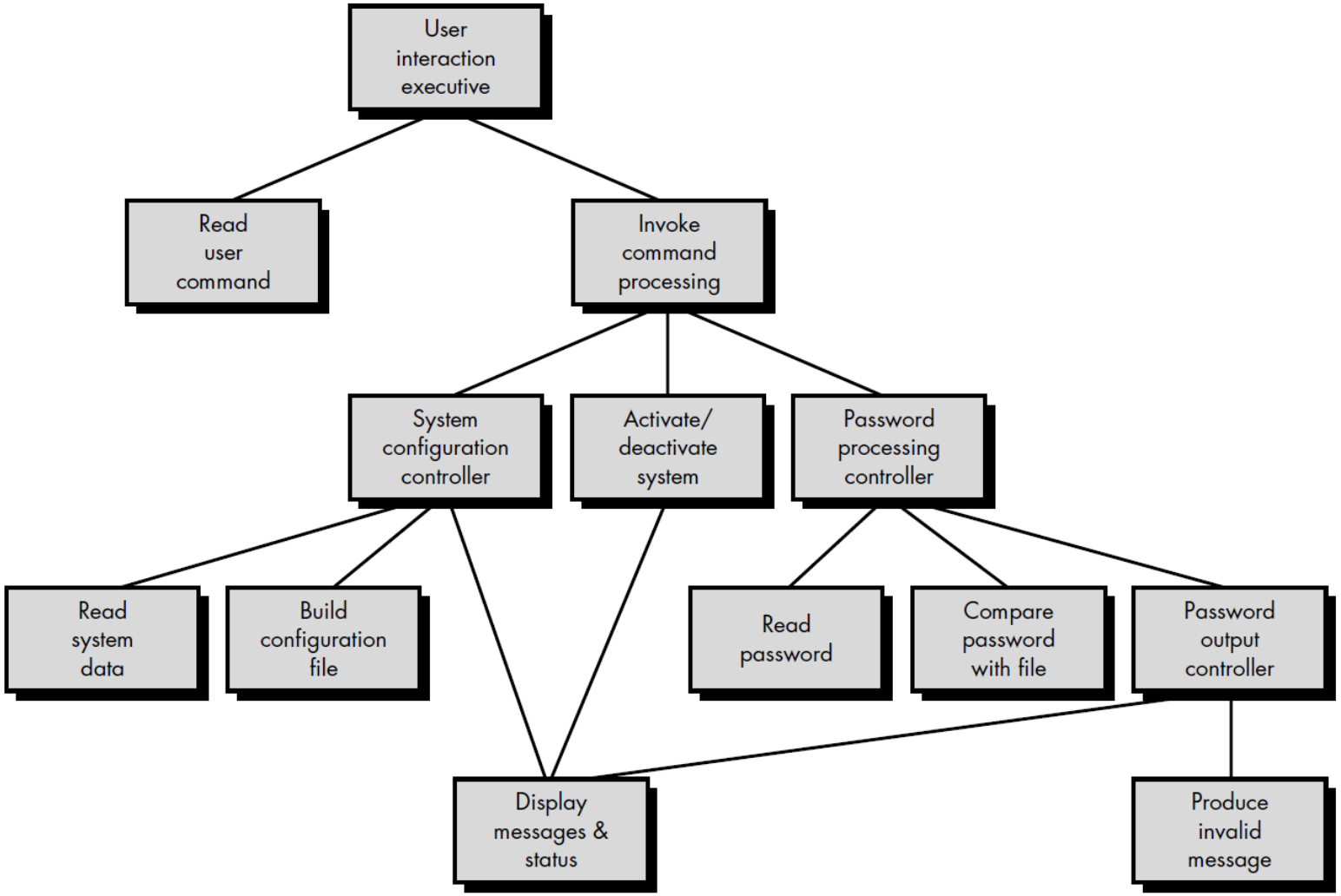
Transaction Mapping



DFD for "User Interaction System"



SC for User Interaction System (first iteration)



SC for User Interaction System (second iteration)